

The Nine Days

Assignment #8, Narrative Engine Prototype. Rex Machina capstone.

A text engine where a dog walks home across nine days. Two agents. One records what you did. One narrates what happens next. They share a JSON ledger and nothing else, so the narration reflects everything you have done, not just the last thing you typed.

Run it with `python scripted.py`. Nothing to install.

1. The world

Acts 1 and 2 of my capstone, which the game has never had in playable form. You are a shelter dog. The family gave you away and you are walking back to them. Nine days: a gate that does not latch, an alley with something on a leash coming round the corner, a rail yard, two nights in a boxcar, an hour of water on the left, a hand at a diner back door, a highway verge, and a street you know.

The narration never tells you how you feel. It tells you what your body is doing and what is in front of you, and it says it differently depending on what you did earlier.

2. What the ledger tracks

```
{ "day": 8,
  "condition": { "state": "hurt", "since_day": 4 },
  "hunger": 0,
  "trust_in_humans": -2,
  "facts": [ { "fact": "escaped_the_gate", "day": 0 },
             { "fact": "hid_from_the_leash", "day": 1 },
             { "fact": "limped_first", "day": 4 },
             { "fact": "refused_the_hand", "day": 6 } ],
  "places_seen": [ "shelter_yard", "alley", "rail_yard", "boxcar" ],
  "flags": {} }
```

Every fact carries the day it was recorded, so the narration can say when something happened and not just that it did.

`condition` carries the injury and the day it began. `hunger` runs 0 to 4 and `trust_in_humans` runs -2 to 2, so a decision about a stranger's hand on day 6 is still shaping how day 8 reads.

This is not a structure invented for the assignment. GDD §4 already specifies `journey_memory` and `condition` in exactly this shape for the Chronicler. The difference is that here it is written by play instead of authored in advance, which is the piece the capstone needs.

3. The loop

```
your typed action
  |
  v
TRACKER agent      -> a JSON patch: facts, flags, hunger, trust, condition
  |
  v
THE LEDGER'S RULES  every field checked before anything is written
  |                  refusals are logged with a reason
  v
NARRATOR agent      <- the whole ledger, every turn
  |
  v
CONSISTENCY CHECK  banned words, third person, invented names,
  |                  the wrong leg, claims the ledger has ruled out
  v
evidence/session.md
```

The tracker never sees the narration and the narrator never sees the patch. The tracker has one job: turn what you did into a change to the ledger. That is what the assignment means by tracking what a player does, not what they say.

Nothing the tracker proposes is trusted. The ledger refuses a fact that contradicts one it already holds, a duplicate, a malformed id, a delta larger than the world allows, an unknown condition, a non-boolean flag, and a fact recorded from an action that was phrased as a refusal. Every refusal is kept with its reason. `evidence/guards.md` drives all seven rules with input they reject, so each one can be watched working.

4. Reactive dialogue

Two ledgers, identical except for one decision on day 3. Same day-6 scene, same typed action, both narrated. Anything that differs came from the ledger.

The dog who took the food. `trust_in_humans 1`, hunger 0.

You limp toward the propped door. A hand appears, sets down a plate of scraps. The hand withdraws. You eat standing up, ready to leave.

The dog who refused the hand. `trust_in_humans -1`, hunger 2.

The door is propped open with a brick. A hand appears above the threshold, setting down a plate, scraps of meat and bread crusts still warm. You stop at the threshold, your back leg throbbing. The hand withdraws.

She is hungrier in the second one and she still does not cross. `python ab_test.py` regenerates both. Earlier runs are kept in `evidence/`, and the contrast holds in every one of them with different words each time.

5. Consistency across nine turns

Nine turns, nine facts, zero consistency flags in the shipped run. The check runs on every line and reports whether or not anything fires, so the log shows the same thing on a clean turn as on a broken one.

The narration stayed with the ledger where it would have been easy not to. Turn 8, five turns after the injury was recorded:

Your left hind leg pulls tight with each step, the way it has since day four.

6. The moment it surprised me

The line above. I never asked the narrator to give a date. It read `{"fact": "limped_first", "day": 4}` out of the ledger and wrote "since day four" on its own, five turns later, in a sentence about walking beside a road.

It is worth reporting because the ledger almost lost that date. On turn 8 the tracker changed the condition from limping to hurt, and my code stamped the new state with the current day, so `condition.since_day` jumped from 4 to 7. The narration still got it right, because the `limped_first` fact carries its own day. The field lost the date and the fact kept it. That is a reason to record events rather than states. The bug is fixed and there is a test for it.

7. What four live runs fixed

Each run is kept in `evidence/`. The failures are the part worth reading.

Run 1: the tracker recorded the opposite of what I did. I typed "the hand puts a plate down, back away from it and wait" and it wrote `took_food_from_stranger`. The narrator then built on it: "You've already taken food from a stranger's fingers once today." That is the assignment's core requirement failing in one turn. There is now an engine rule that swaps a fact for its opposite when the action reads as a refusal, and it logs the swap.

Run 1 also let the tracker decide when a day ended, and it spent three turns on day 0, so by turn 8 the narrator was in the boxcar while the script had reached the street. The calendar is engine state now.

Run 2: the leg moved. Back leg on turn 4, front on turn 5, left front on turn 7. GDD §4 names the back leg, so the narrator is told which leg it is and the checker flags the others.

Run 3: my checker scored one true catch, two false positives and a miss. It missed "front left leg" because it matched a fixed phrase list and the model wrote the words in another order. It flagged "Grease" as an invented name because a hyphen split a capitalised token. And it called a line contradictory for saying "the plate", when a plate on the ground is scenery. All three are fixed, each with a regression test.

Run 4 is the one in this archive. Nine turns, no flags, no refusals.

The Assignment 6 feedback said a rule with unit-test coverage and no live trigger is a rule nobody has watched work. The engine refused nothing in run 4, which is exactly that situation, so `guards.py` drives every rule with input it rejects. The first case is the real patch the tracker produced in run 1.

8. Running it

<code>python scripted.py</code>	the nine-turn session, recorded turns, no key
<code>python scripted.py --live</code>	real calls against claude-haiku-4-5
<code>python ab_test.py [--live]</code>	the same scene under two ledgers
<code>python guards.py</code>	every engine rule, firing. No key, no model
<code>python play.py</code>	type your own actions. Needs a key
<code>python tests/test_dm.py</code>	47 assertions
<code>python run_live.py</code>	all of it, one command

Python 3.8 or newer. To run it live, supply your own key in the same terminal:

```
PowerShell: $env:ANTHROPIC_API_KEY = "your-key"
cmd.exe:    set ANTHROPIC_API_KEY=your-key
bash or zsh: export ANTHROPIC_API_KEY=your-key
```

No key is stored anywhere in this archive and none should ever be.

9. Known limits

Zero consistency flags is a smaller claim than it looks. The checker caught real breaks in runs 2 and 3, and it has also been wrong in both directions. A clean run means the narration passed my checks, not that it was consistent.

The checks are regular expressions. They catch a banned word, a name, the wrong leg and a handful of contradiction phrases. They cannot see that run 1 narrated her curled against a dumpster while the ledger had her in a boxcar, which is the most obvious break in this whole archive and the one nothing automated found.

The tracker is one call and keeps no memory of its own reasoning. It reads the ledger and the action, nothing else. That keeps it cheap, and it means a subtle action tends to get recorded as the nearest known fact.

Nine turns is above the five the assignment asks for and it is still one session. The failures above appeared at a rate of about one per run, so a twenty-turn session would likely surface a class of problem this one did not.